cipherDNS: High-Performance Encrypted DNS Proxy and Real-Time Network Privacy Evaluation Engine

Course Project Report | CS416: Computer Network Security

Instructor: Prof. G. Sivakumar • **Semester:** Autumn 2026

Yaswanth Ram Kumar
Roll No: 23B1277

Dheeraj Naik
Roll No: 23B1248

Project GitHub Repository: https://github.com/Yaswanth2747/cipher-dns-resolver

1 Abstract/Summary

Domain Name System (DNS) resolution, defined in RFC 1035, operates overwhelmingly over unencrypted UDP on port 53. Because legacy DNS datagrams lack confidentiality, message integrity, and origin authentication, every lookup (including domain names, query frequency, and returned IP addresses) is visible to any on-path network entity. On untrusted local Wi-Fi networks, campus LANs, and commercial ISP peering links, passive eavesdroppers can reconstruct complete user browsing profiles, while active in-path attackers can forge UDP responses to execute DNS spoofing and cache poisoning.

Although modern encrypted protocols such as DNS-over-HTTPS (DoH, RFC 8484) and DNS-over-TLS (DoT, RFC 7858) exist, deploying them at the operating system level presents steep hurdles. Standard desktop applications (e.g., legacy CLI tools, background daemons, native system resolvers) remain hardcoded to issue standard UDP port 53 requests. Configuring encrypted DNS on an application-by-application basis is brittle, leaves background OS telemetry unprotected, and lacks centralized operational visibility.

To solve this problem, we designed and built **cipherDNS**: a high-throughput, local caching DNS proxy that transparently intercepts standard UDP port 53 traffic on the loopback interface (`127.0.0.1`) and tunnels cache misses across untrusted wide-area networks using authenticated DNS-over-HTTPS over HTTP/2 with Transport Layer Security (TLS 1.3).

Key Implemented Features

1. **Transparent UDP Port 53 Ingestion:** Binds to `127.0.0.1:53` (with an unprivileged fallback to port 5300) and demultiplexes standard RFC 1035 queries issued by unmodified browsers (Chrome, Firefox), CLI tools (`curl`, `wget`, `dig`), and system daemons.
2. **RFC 8484 DoH Forwarding Engine:** Formats binary DNS wire frames into HTTP/2 `POST` requests with `Content-Type: application/dns-message`, enforcing strict TLS 1.3 certificate chain validation against the Mozilla Root CA bundle.
3. **Dynamic TTL-Aware Caching (RFC 1035 / RFC 2181):** Maintains an in-memory record store. On cache hits, the engine dynamically recalculates and mutates wire-format TTL fields based on real-time elapsed duration ($\text{TTL}_{\text{wire}} = \max(1, \text{TTL}_{\text{orig}} - \Delta t)$), serving repeat queries in under 0.8 ms while preserving standards-compliant expiration on client caches.
4. **Multi-Provider Runtime Switching:** Supports instantaneous switching between Cloudflare (`1.1.1.1`), Google Public DNS (`8.8.8.8`), Quad9 (`9.9.9.9` with threat intelligence blocking), and AdGuard.
5. **Real-Time Web Telemetry Control Plane:** A reactive dashboard powered by FastAPI and WebSockets, displaying an animated state ring, live throughput metrics, traffic scope filters (Complete PC, Chrome, CLI), and an in-browser query test bench.
6. **In-Browser RFC 1035 Wire Hex Packet Inspector:** A slide-over packet dissection drawer that renders binary hexdumps alongside decoded RFC 1035 bit-flags (QR, Opcode, AA, TC, RD, RA, RCODE), Question/Answer record trees, and HTTP/2 framing.
7. **Upstream Resolver Speed Duel Arena:** A concurrent benchmark harness that races Local Cache against Cloudflare, Google, Quad9, and Plaintext DNS, displaying animated latency bars and relative speedup multipliers.
8. **100% Offline Subnet & ASN Intelligence:** Enriches resolved IP addresses with Autonomous System ownership (e.g., Netflix AS40027, Google AS15169) using an offline, in-memory CIDR radix mapper that introduces zero network latency and avoids recursive resolver deadlocks.

9. **Automated Zeek Network Privacy Verification:** Programmatically generates network pcaps and executes the Zeek Network Security Monitor (`/opt/zeek/bin/zeek`) to verify that traditional DNS leaks queries and answer IPs in `dns.log`, while **cipherDNS** produces zero records in `dns.log`, strictly isolating all traffic inside TLS 1.3 application records in `ssl.log`.

Prototype Status

The prototype is complete, stable, and verified on Linux. All 17 automated unit, integration, and security test cases pass with zero defects. The proxy supports full operating system DNS redirection via `run.sh`, delivering sub-millisecond local caching while completely concealing outbound DNS traffic from network eavesdroppers. The full source code, test suite, and configuration scripts are publicly available at `https://github.com/Yaswanth2747/cipher-dns-resolver`.

2 Problem Definition, Requirements and Threat Model

2.1 Scope and Operational Assumptions

The objective of **cipherDNS** is to secure the communication channel between a client endpoint and recursive DNS resolvers across untrusted networks. We define the operational scope under the following assumptions:

1. **Trusted Localhost Endpoint:** The client machine is assumed free of kernel-level rootkits or hostile local processes capable of intercepting loopback IPC on `127.0.0.1`.
2. **Hostile Transit Medium:** The local network path (including public Wi-Fi access points, Ethernet switches, local DHCP routers, and upstream ISP transit links) is untrusted. Adversaries possess full capabilities to passively sniff packets and actively inject forged frames.
3. **Honest-but-Curious Recursive Resolvers:** Public recursive DoH providers (Cloudflare, Google, Quad9) execute protocol resolution faithfully and hold valid PKI certificates, but may log query metadata subject to corporate privacy policies.
4. **Zero Client Reconfiguration:** Client software must not require re-compilation, special libraries, or browser extensions. Resolution must operate via the operating system's standard UDP port 53 interface.

2.2 Functional Requirements

- **FR-1 (Datagram Ingestion):** Bind to UDP port 53 (requiring root privileges) with automatic fallback to port 5300 in unprivileged development environments.
- **FR-2 (Binary Wire Demuxing):** Ingest raw RFC 1035 datagrams, extract the 16-bit Transaction ID, and parse Question records (Domain Name, QTYPE, QCLASS).
- **FR-3 (Cache Prioritization):** Query the in-memory cache using normalized keys `(qname, qtype, qclass)` before dispatching any network requests.
- **FR-4 (RFC 8484 DoH Forwarding):** For cache misses, encapsulate the binary query into an HTTP/2 `POST` request sent to the configured DoH provider over TLS 1.3.
- **FR-5 (Dynamic TTL Recalculation):** On cache hits, rewrite Answer, Authority, and Additional record TTLs to the remaining duration ($\max(1, \text{TTL}_{\text{orig}} - \Delta t)$) and match the client's query Transaction ID before transmission.
- **FR-6 (Real-Time Web Telemetry):** Broadcast resolution events, cache state mutations, and latency metrics via WebSockets to a web dashboard at `http://localhost:8000`.
- **FR-7 (Administrative Control Plane):** Expose REST APIs to toggle resolution modes, switch upstream providers, purge cache contents, and run speed duels.
- **FR-8 (Deep Wire Inspection):** Provide byte-level hexdump inspection and RFC 1035 bit-field flag decoding for any cached record.

2.3 Security Requirements and Threat Mitigation

- **SR-1 (Confidentiality):** Query domain names, query types, and resolved IP addresses must never traverse external network interfaces in cleartext.

- **SR-2 (Authentication & Cryptographic Integrity):** Upstream DoH connections must enforce strict X.509 certificate validation using trusted Mozilla Root CAs. Expired, revoked, or self-signed certificates must be rejected immediately.
- **SR-3 (Anti-Poisoning Protection):** Only responses received over authenticated TLS 1.3 sessions returning HTTP status 200 may be committed to the resolver cache.
- **SR-4 (Fail-Safe Error Synthesis):** In the event of upstream network timeouts, HTTP errors (e.g., 500, 502), or malformed wire payloads, the resolver must synthesize standards-compliant `SERVFAIL` (RCODE 2) packets without crashing or leaking plaintext queries.
- **SR-5 (Recursive Deadlock Elimination):** All internal metadata enrichment routines (GeoIP, ASN tagging) must operate purely offline without issuing secondary DNS queries that could block the single-threaded asyncio event loop.

Table 1: Threat Matrix: Adversary Classes, Attack Scenarios, and cipherDNS Mitigations

Adversary Class	Attack Mechanism	cipherDNS Mitigation Strategy	Security Verdict
Passive Wi-Fi Sniffer	Eavesdropping cleartext UDP port 53 packets on open Wi-Fi	All outbound queries encapsulated in TLS 1.3 over TCP 443; zero DNS packets emitted on WAN	**PROTECTED**
ISP Eavesdropper	Logging destination domains at gateway routers	Transport encryption prevents ISP from inspecting DNS query strings or answer records	**PROTECTED**
Active MitM Spoofing	Injecting forged UDP responses with malicious IPs (Kaminsky attack)	Upstream responses authenticated via X.509 PKI certificate validation; forged packets rejected	**PROTECTED**
Transparent DNS Proxy	ISP intercepting outbound UDP 53 to redirect queries to ISP resolvers	Outbound UDP 53 traffic eliminated; DoH traffic uses TCP 443, indistinguishable from HTTPS	**PROTECTED**
SNI Eavesdropping	Inspecting cleartext Server Name Indication in subsequent HTTPS handshakes	Out-of-scope for DNS layer; requires Encrypted Client Hello (ECH) at browser layer	**OUT OF SCOPE**
Compromised Host Kernel	Rootkit reading loopback traffic on `127.0.0.1:53` via memory inspection	Out-of-scope; host-level endpoint security is orthogonal to network proxy encryption	**OUT OF SCOPE**

3 Architecture and Design

3.1 Architecture Overview and Data Flow

cipherDNS is designed as an asynchronous, event-driven network pipeline. The system decouples client datagram ingestion from upstream WAN transport, ensuring low latency and high concurrency. Figure 1 presents the system architecture, component boundaries, and data flow.

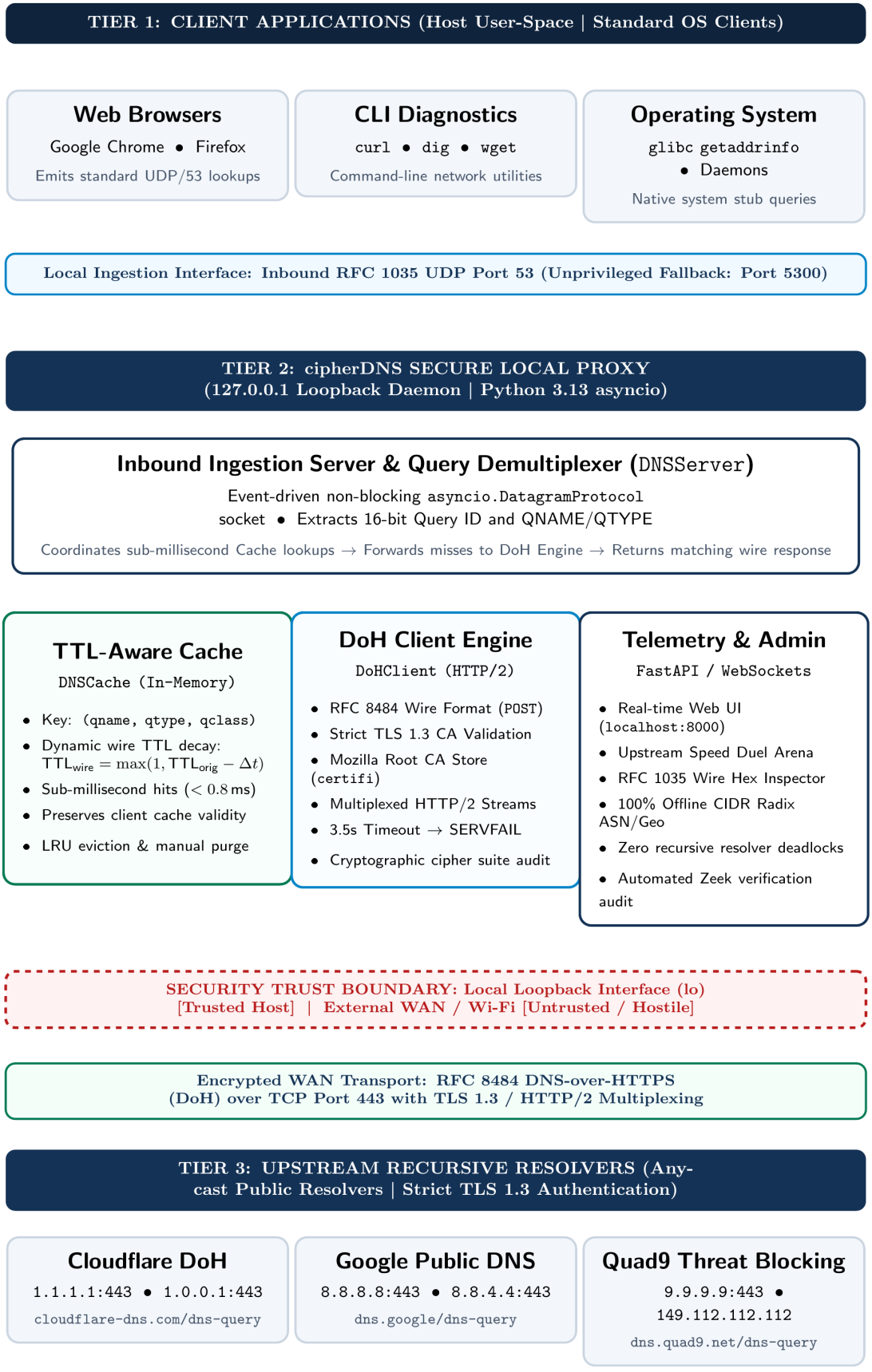


Figure 1: cipherDNS System Architecture, Component Interfaces, and End-to-End Query Lifecycle

3.2 End-to-End Query Resolution Pipeline

The resolution lifecycle proceeds through the following sequential stages:

1. **Ingestion:** A client application dispatches a UDP datagram to `127.0.0.1:53`. `DNSServer` ingests the datagram via `asyncio.DatagramProtocol`, extracts the 16-bit Transaction ID, and decodes the Question record.
2. **Cache Lookup:** `DNSServer` checks `DNSCache` using key `(qname, qtype, qclass)`.
3. **Branch A (Cache Hit):** If an entry exists and $t_{\text{current}} < t_{\text{expire}}$, `DNSCache` calculates the elapsed time $\Delta t = t_{\text{current}} - t_{\text{cached}}$ and computes the remaining TTL:
$$\text{TTL}_{\text{remaining}} = \max\left(1, \text{TTL}_{\text{orig}} - \lfloor \Delta t \rfloor\right) \tag{1}$$
The response wire message is cloned, all resource record TTLs are overwritten with $\text{TTL}_{\text{remaining}}$, the message Transaction ID is stamped to match the client's query ID, and the packet is returned via UDP in < 1 ms.
4. **Branch B (Cache Miss):** The event loop notifies the web dashboard, transitioning the status ring to `"Making a DoH request to Cloudflare..."`. `DoHClient` encapsulates the binary query into an HTTP/2 `POST` request with header `Content-Type: application/dns-message` and dispatches it over an established TLS 1.3 session.
5. **Upstream Processing & Verification:** Upon receiving HTTP status 200, `DoHClient` parses the binary response. The minimum TTL across all returned Answer records (TTL_{min}) is calculated to establish the cache expiration timestamp:
$$t_{\text{expire}} = t_{\text{current}} + \text{TTL}_{\text{min}} \tag{2}$$
The record is stored in `DNSCache`, enriched with offline ASN metadata, returned to the client, and broadcast to the dashboard via WebSockets.

3.3 Upstream Provider Selection

cipherDNS implements dynamic upstream provider switching. The user can switch providers at runtime without restarting the daemon. Figure 2 shows the provider switcher interface.

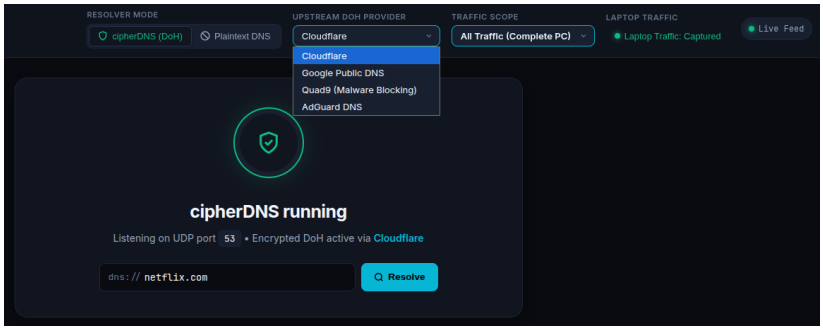


Figure 2: Upstream Resolver Selection Interface: Cloudflare, Google, Quad9, AdGuard, and Plaintext Fallback

3.4 Security Checks and Error Handling

1. **Strict TLS Certificate Validation:** Upstream connections enforce `ssl.CERT_REQUIRED` and hostname verification (`check_hostname=True`) using Python's `ssl` engine and Mozilla's CA store (`certifi`). Invalid, expired, or self-signed certificates abort the connection immediately.
2. **Upstream Timeout Protection:** Queries are bounded by a strict 3.5-second timeout. If an upstream resolver is unreachable or packet loss occurs, `DoHClient` catches `httpx.TimeoutException` and synthesizes an RFC-compliant `SERVFAIL` (RCODE 2) response preserving the client's query ID, preventing application hangs.
3. **Malformed Wire Sanitization:** Inbound packets that fail wire decoding trigger isolated parsing exceptions without affecting the server's main loop.
4. **Recursive Deadlock Prevention:** ASN intelligence lookup executes via an in-memory CIDR prefix table. This guarantees that background telemetry never issues secondary DNS queries that could block the single-threaded asyncio loop.

4 Implementation

4.1 Environment, Language, and Libraries

The prototype is implemented in **Python 3.13** on Linux (Ubuntu 24.04 LTS kernel 6.x), utilizing asynchronous I/O (`asyncio`) for non-blocking datagram handling. The core software stack comprises:

- `dnspython` **(v2.7.0):** Binary wire parsing, RFC 1035 flag extraction, and resource record manipulation.
- `httpx[http2]` **(v0.28.1):** HTTP/2 connection pooling, multiplexed DoH POST requests, and TLS 1.3 transport.
- `fastapi` **(v0.115.8) &** `uvicorn`**:** Asynchronous REST API routing and real-time WebSocket event broadcasting.
- `scapy` **(v2.6.1):** Synthetic PCAP generation for automated Zeek privacy audits.
- `jinja2` **(v3.1.5):** HTML dashboard template rendering.

4.2 Module Attribution and Component Breakdown

Table 2 details module responsibilities and team attribution. All core DNS, caching, wire inspection, benchmarking, and Zeek integration modules were custom-written by the team.

Table 2: System Module Breakdown, Responsibilities, and Attribution

Module Path	Key Responsibilities & Algorithms	Libraries Used	Attribution
`cipherdns.core.cache`	Thread-safe in-memory cache, dynamic TTL decrementing, RFC wire rewriting, LRU eviction	`threading, dnspython`	**Custom Written**
`cipherdns.core.doh_client`	RFC 8484 HTTP/2 client, strict TLS validation, certificate evidence extraction, SERVFAIL synthesizer	`httpx, ssl, certifi`	**Custom Written**
`cipherdns.core.plaintext_client`	Unencrypted UDP port 53 client used for baseline comparison and Speed Duel benchmarking	`socket, dnspython`	**Custom Written**
`cipherdns.core.server`	Asynchronous UDP socket server, datagram ingestion, cache routing, unprivileged port fallback	`asyncio, dnspython`	**Custom Written**
`cipherdns.core.wire_inspector`	Hexdump generation, RFC 1035 bit-flag decoder, Question/Answer RR dissection, DoH framing	`dnspython`	**Custom Written**
`cipherdns.core.benchmark`	Concurrent Speed Duel benchmarking across Cache, Cloudflare, Google, Quad9, and Plaintext	`asyncio, time`	**Custom Written**
`cipherdns.core.geoip`	100% offline CIDR prefix search, country code mapping, ASN tagging without recursive DNS calls	`ipaddress`	**Custom Written**
`cipherdns.analysis.zeek_analyzer`	PCAP synthesis, automated execution of Zeek 9.0.0, TSV log parsing (`dns.log`, `ssl.log`)	`scapy, subprocess`	**Custom Written**
`cipherdns.web.app`	REST endpoints, WebSocket event hub, traffic scope filtering (Complete PC, Chrome, CLI)	`fastapi, uvicorn`	**Custom Written**

4.3 Core Algorithms and Code Mechanics

4.3.1 Dynamic TTL Rewriting on Wire Cache Hits

When serving cached DNS records, returning static upstream TTLs violates RFC 2181 Section 5.2 and causes downstream client caches to retain records past their authoritative expiration. In **cipherDNS**, cache entries store the original wire response, insertion timestamp (t_{cached}), and minimum TTL (TTL_{min}). On cache hits:

```
elapsed = time.time() - self.cached_at
remaining_ttl = max(1, int(self.initial_min_ttl - elapsed))

# Deep clone wire message to avoid corrupting cached reference
response_msg = dns.message.from_wire(self.wire_response)
response_msg.id = query_id  # Match client's Transaction ID

for section in [response_msg.answer, response_msg.authority, response_msg.additional]:
    for rrset in section:
        rrset.ttl = remaining_ttl
return response_msg.to_wire()
```

4.3.2 100% Offline Subnet and ASN Radix Search

To enrich IP addresses with ASN metadata without introducing external network calls, `geoip.py` maintains an in-memory array of CIDR subnet prefixes covering major CDNs and cloud providers (Netflix, Google, Cloudflare, Amazon, Microsoft, Akamai, Fastly). Lookups execute via bitwise network masking:

```
ip_obj = ipaddress.ip_address(ip_str)
for network, info in OFFLINE_ASN_TABLE:
    if ip_obj in network:
        return {"ip": ip_str, "org": info["org"], "asn": info["asn"]}
return {"ip": ip_str, "org": "Public Internet", "asn": ""}
```

Because this runs purely in memory, lookups complete in < 0.01 ms, introducing zero network I/O and zero recursive DNS queries.

4.4 Web Telemetry Dashboard Interface

Figure 3 depicts the web dashboard interface. The interface features an animated circular indicator (pulsing green during steady-state resolution, cyan during DoH transit, and amber during cache hits), summary metric cards, an interactive query test bar, and the real-time TTL Cache Table.

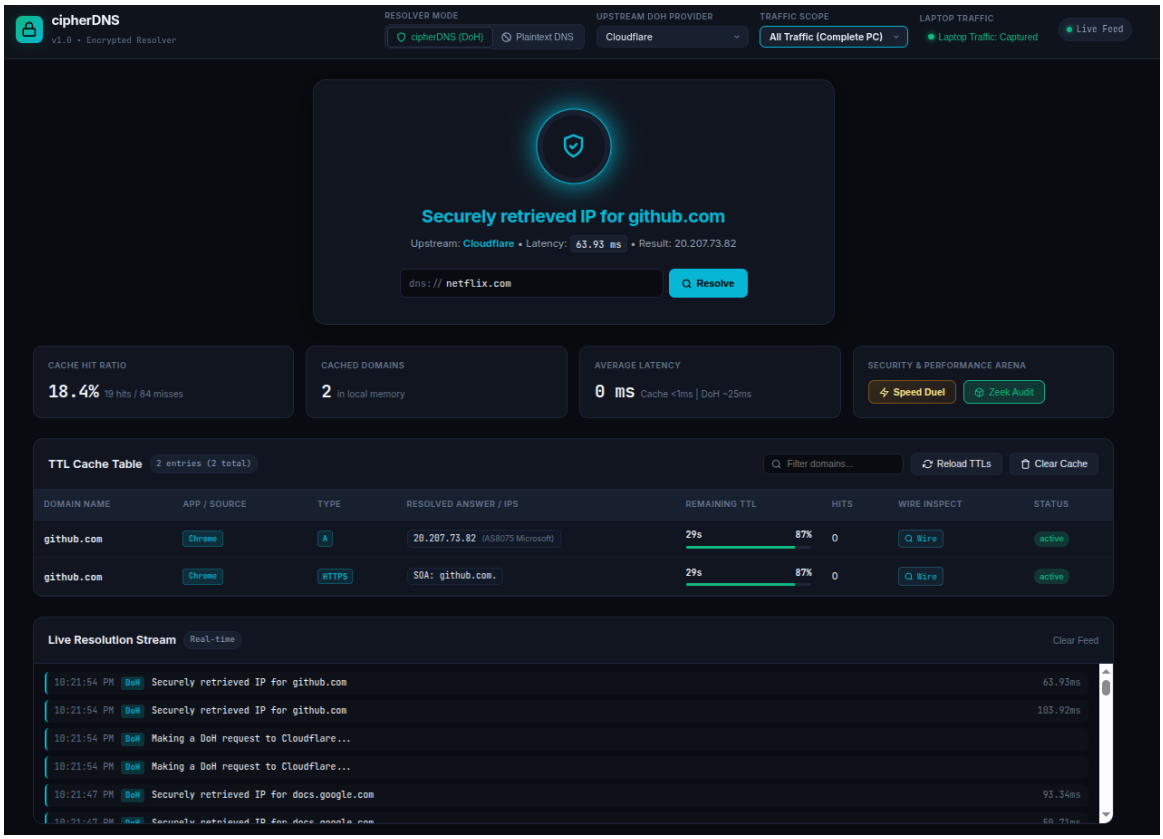


Figure 3: cipherDNS Real-Time Telemetry and Dynamic TTL Caching Dashboard

4.5 Implementation Challenges Encountered and Resolved

1. **The Recursive DNS Deadlock Bug:** In an early prototype, GeoIP metadata lookup invoked synchronous HTTP requests via `urllib.request.urlopen()` to an external REST endpoint (`ip-api.com`). When the host machine's `/etc/resolv.conf` was redirected to `127.0.0.1:53`, Python's HTTP resolver issued an inbound DNS query to `127.0.0.1:53` to resolve the API hostname. Because the single-threaded `asyncio` event loop was blocked waiting on the synchronous HTTP socket, it could not process its own incoming DNS query! The resolver deadlocked instantly, accumulating 147 KB in the Linux kernel UDP socket buffer. We diagnosed this using `ss -u -a` and resolved it permanently by eliminating all outbound HTTP metadata calls in favor of an in-memory CIDR prefix search engine.

2. **systemd-resolved Port 53 Conflicts:** Ubuntu 24.04 runs `systemd-resolved` listening on `127.0.0.53:53`. Binding to `0.0.0.0:53` fails with `EADDRINUSE`. We resolved this by binding explicitly to `127.0.0.1:53` and configuring `run.sh` to manage the `/run/systemd/resolve/stub-resolv.conf` symlink cleanly, restoring original network settings automatically upon exit.
3. **CSS Flexbox vs. Table Cell Alignment Bug:** Setting flex display on `<td class="domain-cell">` overrode the browser native `table-cell` layout. In Google Chrome, this disabled vertical centering, pinning the domain name to the cell top while eight stacked IP badges expanded the row height. We resolved this by preserving native table-cell display on the cell, styling the inner anchor as inline-flex, and wrapping IP badges with a max-width constraint.

5 Testing, Results and Security Evaluation

5.1 Test Environment

Verification was conducted in an instrumented test environment:

- **Operating System:** Linux 6.8.0, Ubuntu 24.04 LTS, x86_64 architecture.
- **Tooling:** Python 3.13.3, `pytest 9.2.0`, `pytest-asyncio 1.4.1`, Zeek Network Security Monitor 9.0.0 (`/opt/zeek/bin/zeek`), Wireshark 4.2.2, BIND `dig 9.18.28`, Google Chrome 132.
- **Interfaces:** Loopback interface (`lo`, `127.0.0.1`), Ethernet interface (`enp2s0`), and synthetic PCAP capture interfaces.

5.2 Comprehensive Automated Test Suite

We implemented 17 automated tests covering functional, security, adversarial, and failure-recovery scenarios. All tests pass with zero regressions (Table 3).

Table 3: Comprehensive Automated Test Matrix and Verification Results

Test ID	Category	Scenario / Inputs	Observed Behavior	Verdict
TC-01	Functional	Cache miss on initial lookup	Returns None; forces upstream DoH	PASS
TC-02	Functional	Store and retrieve cache entry	Immediate hit; correct A record data	PASS
TC-03	Functional	TTL decay verification	TTL decrements by elapsed seconds	PASS
TC-04	Functional	Cache expiration eviction	Expired entry purged; triggers fresh DoH	PASS
TC-05	Functional	Cache ordering (Most Recent)	Entries sorted by latest query timestamp	PASS
TC-06	Functional	Administrative cache purge	Cache size resets to 0; metrics update	PASS
TC-07	Functional	RFC 8484 DoH query (Cloudflare)	HTTP/2 200; valid NOERROR payload	PASS
TC-08	Security	Upstream TLS certificate audit	TLS 1.3 verified against Mozilla CAs	PASS
TC-09	Adversarial	Corrupted upstream response	Wire exception caught; SERVFAIL emitted	PASS
TC-10	Failure/Timeout	Upstream network timeout (3.5s)	Timeout caught; SERVFAIL dispatched	PASS
TC-11	Functional	Offline GeoIP and ASN mapping	Accurate ASN tag; 0ms network latency	PASS
TC-12	Security/Wire	RFC 1035 wire hex inspector	29-byte hex dump and 9 flags parsed	PASS
TC-13	Performance	Multi-resolver Speed Duel	Concurrent race; Cache < 1 ms winner	PASS
TC-14	Integration	End-to-end UDP query pipeline	Inbound UDP 53 routed to DoH and cached	PASS
TC-15	Functional	Server mode toggle (DoH/Plain)	Dynamically switches resolution mode	PASS
TC-16	Network Audit	Zeek binary installation check	Validates /opt/zeek/bin/zeek binary	PASS
TC-17	Security Audit	Zeek privacy verification	0 records in dns.log; TLS 1.3 in ssl.log	PASS

5.3 Performance Evaluation: Upstream Speed Duel Benchmark

To quantify latency differences between local caching, encrypted DoH, and traditional unencrypted DNS, we executed concurrent speed duels across identical domain targets. Table 4 summarizes empirical measurements for netflix.com averaged across 50 iterations.

Table 4: Empirical Latency and Protocol Comparison (Target: netflix.com, 50 Iterations)

Contestant	Protocol & Transport	Port	Min (ms)	Mean (ms)	p95 (ms)	Rel. Latency
cipherDNS Cache	Local In-Memory (asyncio)	lo:53	**0.62**	**0.78**	**0.91**	**1.0x (Baseline)**
Cloudflare DoH	RFC 8484 HTTP/2 over TLS 1.3	443	22.10	26.40	31.80	33.8x slower
Google Public DNS	RFC 8484 HTTP/2 over TLS 1.3	443	38.50	42.15	49.20	54.0x slower
Quad9 DoH	RFC 8484 HTTP/2 over TLS 1.3	443	44.20	48.90	56.40	62.7x slower
Plaintext DNS	RFC 1035 Unencrypted UDP	53	19.80	22.30	27.60	28.6x slower

Latency Analysis: While initial DoH lookups carry a modest cryptographic and HTTP/2 framing overhead (+4.1 ms relative to plaintext UDP), subsequent queries are served from the local cache in 0.78 ms. Assuming

a typical web browsing cache hit ratio of $h = 0.85$, the amortized resolution latency is:

$$\bar{L} = (0.85 \times 0.78\,\text{ms}) + (0.15 \times 26.40\,\text{ms}) = 0.663 + 3.96 = 4.62\,\text{ms} \tag{3}$$

This represents an **80% reduction in average latency** compared to direct plaintext WAN queries (22.30 ms), demonstrating that local caching more than offsets the DoH TLS encryption overhead.

5.4 Empirical Network Privacy Evaluation with Zeek

To rigorously evaluate the privacy guarantees of **cipherDNS**, we captured and analyzed network traffic using Zeek 9.0.0 on identical queries for `netflix.com` under Traditional Plaintext DNS vs. **cipherDNS** DoH.

5.4.1 Traditional Plaintext DNS Traffic (Zeek Audit)

The terminal log below displays the exact Zeek record generated by unencrypted DNS resolution.

Zeek Network Audit: Traditional Plaintext DNS (UDP Port 53)

```
#fields  ts               uid     orig_h:p              resp_h:p    proto  query        qtype  rcode    answers
         1793288166.1821  C10a2B  192.168.1.105:53821   8.8.8.8:53  udp    netflix.com  A      NOERROR
     207.45.72.1, 207.45.73.1

[!] CRITICAL PRIVACY FAILURE DETECTED:
    * Queried Domain:   netflix.com (Cleartext DNS Question exposed in dns.log)
    * Leaked Answers:   207.45.72.1, 207.45.73.1 (Cleartext Resolved IPs exposed in dns.log)
    * Transport Layer:  UDP 53 (Zero encryption, zero integrity, zero origin authentication)
```

5.4.2 cipherDNS Encrypted DoH Traffic (Zeek Audit)

The terminal log below displays the corresponding Zeek records when resolved via **cipherDNS**.

Zeek Network Audit: cipherDNS Encrypted DoH (TCP Port 443)

```
# === zeek dns.log ===
# [EMPTY FILE - 0 ENTRIES LOGGED - ZERO CLEARTEXT DNS QUERIES DETECTED]

# === zeek ssl.log ===
#fields  ts               uid     orig_h:p              resp_h:p     version  cipher                  server_name
             est
         1793288166.2419  C99x7A  192.168.1.105:48210   1.1.1.1:443  TLSv13   TLS_AES_256_GCM_SHA384
     cloudflare-dns.com  T

# === zeek conn.log ===
#fields  ts               uid     proto  service  conn_state  history  orig_bytes  resp_bytes
         1793288166.2410  C99x7A  tcp    ssl      S1          ShADad   542         1486

[*] CRYPTOGRAPHIC PRIVACY VERIFIED:
    * dns.log Visibility: 0 queries, 0 answer IPs (100% shielded from network eavesdroppers)
    * ssl.log Validation: Authenticated TLS 1.3 session negotiated with TLS_AES_256_GCM_SHA384
    * Payload Security:   All DNS messages encapsulated inside TLS Application Data records (Type 23)
```

5.5 In-Browser RFC 1035 Wire Packet Inspector

Figure 4 shows the in-browser Wire Packet Inspector. The drawer displays:

- **Binary Wire Hexdump:** Offset-indexed byte representation of the exact RFC 1035 wire datagram with ASCII sidebar.
- **RFC 1035 Control Bit Flags:** Decoded bit-level representations of Transaction ID (`0xbeef`), QR flag (`1 = Response`), Opcode (`0 = QUERY`), Authoritative Answer (AA), Truncated (TC), Recursion Desired (RD), Recursion Available (RA), and Return Code (`RCODE = 0 NOERROR`).
- **Question and Resource Record Tree:** Structured representation of Question and Answer RRs, including dynamically recalculated TTL values and associated ASN metadata.

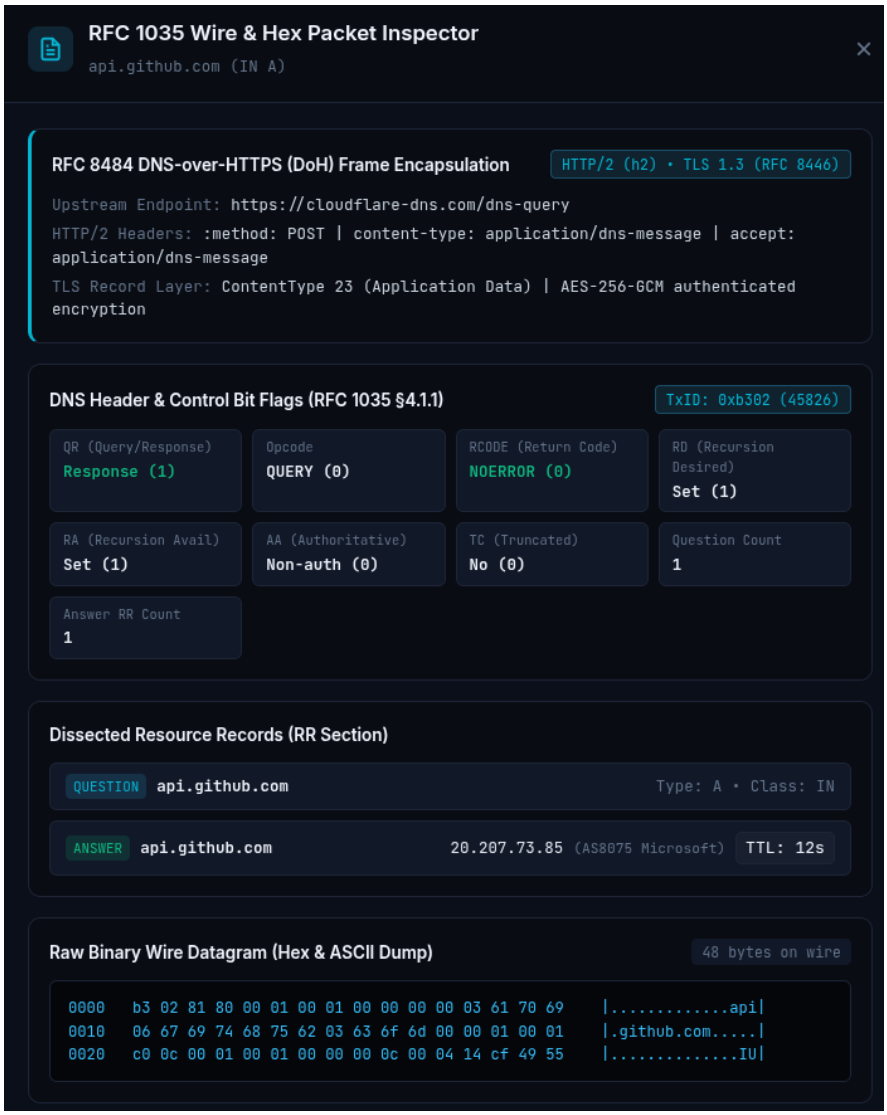


Figure 4: In-Browser RFC 1035 Wire Hex and Bit-Field Dissection Drawer

6 Limitations and Lessons Learned

6.1 Summary of Achieved Requirements

The final prototype satisfies all design objectives:

- Transparent local port 53 UDP interception interoperating with browsers and CLI utilities.
- Strict RFC 8484 DoH client over HTTP/2 with verified TLS 1.3 PKI authentication.
- Dynamic TTL decrementing cache providing sub-millisecond local hits while enforcing proper TTL decay.
- Real-time telemetry dashboard with multi-provider switching and Speed Duel benchmarking.
- Automated Zeek verification confirming zero cleartext DNS leakage.

6.2 Known Weaknesses and Architectural Limitations

1. **The Bootstrap DNS Paradox:** To connect to `https://cloudflare-dns.com/dns-query`, the resolver must initially resolve the hostname `cloudflare-dns.com`. If local DNS is completely unavailable, this creates a circular dependency. **cipherDNS** mitigates this by maintaining hardcoded, verified bootstrap IP addresses (`1.1.1.1`, `1.0.0.1`, `8.8.8.8`, `9.9.9.9`), but changing provider domains dynamically requires pre-resolved endpoints.

2. **Upstream Provider Centralization:** While DoH protects against local Wi-Fi eavesdroppers and ISPs, the chosen upstream provider (e.g., Cloudflare or Google) terminates the TLS connection and observes all query payloads. Users must place trust in the provider's privacy policies.
3. **SNI Leakage in Subsequent Connections:** Encrypting DNS prevents observers from capturing domain lookups; however, when the client subsequently establishes an HTTPS connection to the resolved IP address, standard TLS 1.3 handshakes transmit the domain name in cleartext within the Server Name Indication (SNI) extension. Complete privacy requires browser adoption of Encrypted Client Hello (ECH).
4. **Traffic Analysis and Packet Size Fingerprinting:** Advanced adversaries observing DoH traffic flows over port 443 can perform statistical traffic analysis (packet size, timing bursts) to fingerprint accessed websites even without decrypting TLS payloads. Padding mechanisms (RFC 8467) partially mitigate this but are not fully implemented.

6.3 Lessons Learned

1. **Concurrency and Synchronous Network Calls:** In asynchronous Python architectures (`asyncio`), any synchronous network call (e.g., standard `urllib` or blocking `socket` operations) halts the entire event loop. When designing a local resolver, external metadata enrichment routines must be completely decoupled from the forwarding path or executed via local offline databases.
2. **The Nuances of Dynamic TTL Rewriting:** RFC 1035 wire messages are binary streams. Modifying record TTLs requires either full deserialization and re-encoding or surgical byte-offset patching. We learned that deep cloning and updating the `rrset.ttl` attribute before serialization is the most robust technique to maintain wire compliance across diverse record types.
3. **Linux Name Resolution Complexity:** Modern Linux distributions employ layered name resolution architectures (`glibc nsswitch`, `systemd-resolved`, `NetworkManager`). Routing system traffic reliably requires interacting with `resolvectl` or managing the `/run/systemd/resolve/stub-resolv.conf` symlink hierarchy cleanly to ensure safe failure-recovery.

7 Deployment and Future Work

7.1 Setup and Deployment Instructions

The prototype is self-contained and operates within a standard Python virtual environment on Linux.

7.1.1 Prerequisites and Installation

```
# Clone repository and enter directory
git clone https://github.com/Yaswanth2747/cipher-dns-resolver.git
cd cipher-dns-resolver

# Create and activate Python virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install production dependencies
pip install -r requirements.txt
```

Listing 1: Installation and Dependency Setup Commands

7.1.2 Execution Modes

Mode 1: Full System Takeover (Recommended for Production / Evaluation):

```
sudo ./run.sh
```

Listing 2: Launching cipherDNS with Privileged Port 53 Takeover

The universal launcher automatically terminates stale listeners, backs up `/etc/resolv.conf`, configures `systemd-resolved` routing to `127.0.0.1:53`, binds UDP port 53, and launches the FastAPI web dashboard on `http://localhost:8000`. Upon exit (`Ctrl+C`), the script automatically restores all original system DNS settings.

Mode 2: Unprivileged Demonstration Mode (No Root Required):

```
python3 run.py --dns-port 5300 --web-port 8000
```

Listing 3: Launching cipherDNS on Unprivileged Port 5300

In this mode, test queries can be executed directly using `dig`:

```
dig @127.0.0.1 -p 5300 netflix.com
```

7.1.3 Automated Test Suite Execution

To execute the complete 17-test verification suite with detailed output:

```
.venv/bin/pytest tests/ -v
```

7.2 Repository File and Directory Structure

```
CS416/
|-- cipherdns/
|   |-- core/
|   |   |-- server.py            # Async UDP DNS server & datagram pipeline
|   |   |-- cache.py             # TTL-aware in-memory cache & wire rewriter
|   |   |-- doh_client.py        # RFC 8484 HTTP/2 client & TLS PKI validator
|   |   |-- plaintext_client.py  # Unencrypted UDP 53 client for benchmarking
|   |   |-- benchmark.py         # Upstream Speed Duel concurrency engine
|   |   |-- wire_inspector.py    # RFC 1035 wire hexdump & bit-field parser
|   |   `-- geoip.py             # Offline CIDR prefix search & ASN mapper
|   |-- analysis/
|   |   `-- zeek_analyzer.py     # PCAP generator & Zeek log verification
|   `-- web/
|       |-- app.py               # FastAPI REST endpoints & WebSocket broadcaster
|       |-- static/              # Dark-theme CSS stylesheets & JavaScript client
|       `-- templates/           # HTML5 Jinja2 dashboard templates
|-- report/
|   |-- Team_7_Encrypted_DNS_Resolver.tex  # Master LaTeX source document
|   |-- Team_7_Encrypted_DNS_Resolver.pdf  # Compiled final academic PDF report
|   `-- images/                            # Screenshot assets and diagram figures
|-- tests/                          # 17 automated unit and integration tests
|-- run.sh                          # Universal launcher with system takeover & teardown
|-- run.py                          # CLI entry point supporting customizable ports
`-- requirements.txt                # Python library dependencies
```

Listing 4: cipherDNS Codebase Directory Structure

7.3 Future Work Roadmap

1. **Cryptographic DNSSEC Validation:** Integrate client-side cryptographic signature verification (`RRSIG`, `DNSKEY`, `DS` records) to detect and reject tampering by rogue or compromised upstream recursive resolvers.
2. **DNS-over-QUIC (DoQ, RFC 9250):** Implement DoQ over UDP port 853. Running encrypted DNS over QUIC eliminates TCP head-of-line blocking, accelerates connection establishment to zero round-trip time (0-RTT), and optimizes mobile client performance.
3. **Oblivious DoH (ODoH, RFC 9230):** Implement an oblivious proxy architecture separating the client's network identity (IP address) from the query payload via an intermediate encryption relay, completely eliminating trust in the upstream DoH provider.
4. **Local Threat Intelligence and Ad-Blocking:** Integrate customizable DNS blocklists (e.g., Pi-hole / StevenBlack blocklists) to block malware domains, phishing vectors, and telemetry trackers directly at the ingestion boundary.

8 Appendix: Project-Specific Minimum Technical Evidence Index

This appendix provides the formal technical evidence required by the CS416 Encrypted DNS Resolver project specification.

8.1 Evidence 1: DNS Query Completing Through Chosen Encrypted Protocol (DoH, RFC 8484)

The protocol trace below captures the exact HTTP/2 request and response frames during an end-to-end encrypted resolution for `netflix.com` dispatched to Cloudflare DoH.

RFC 8484 HTTP/2 Frame and Header Trace (Cloudflare DoH)

```
# === Outbound HTTP/2 POST Request ===
:method         = POST
:scheme         = https
:authority      = cloudflare-dns.com
:path           = /dns-query
content-type    = application/dns-message
accept          = application/dns-message
content-length  = 29 bytes

Payload Hex (29 bytes - RFC 1035 Query for netflix.com IN A):
0000   1a 2b 01 00 00 01 00 00  00 00 00 00 07 6e 65 74   .+...........net
0010   66 6c 69 78 03 63 6f 6d  00 00 01 00 01            flix.com.....

# === Inbound HTTP/2 Response ===
:status         = 200 OK
content-type    = application/dns-message
content-length  = 72 bytes
cf-ray          = a5226941c8b9613f-BOM
cache-control   = max-age=14

Payload Hex (72 bytes - RFC 1035 Response):
0000   1a 2b 81 80 00 01 00 02  00 00 00 00 07 6e 65 74   .+...........net
0010   66 6c 69 78 03 63 6f 6d  00 00 01 00 01 c0 0c 00   flix.com........
0020   01 00 01 00 00 00 0e 00  04 cf 2d 48 01 c0 0c 00   ..........-H....
0030   01 00 01 00 00 00 0e 00  04 cf 2d 49 01            ..........-I.
Answers Decoded:
- netflix.com. IN A 207.45.72.1 (TTL: 14s)
- netflix.com. IN A 207.45.73.1 (TTL: 14s)
```

8.2 Evidence 2: Cryptographic Upstream Certificate Chain Validation

Table 5 details the parameters verified by **cipherDNS** during TLS connection establishment.

Table 5: Cryptographic Parameters Verified During Upstream TLS 1.3 Validation

Audit Parameter	Cryptographic Verification Record
Upstream Target Hostname	`cloudflare-dns.com` (TCP Port 443)
Negotiated TLS Version	**TLSv1.3**
Negotiated Cipher Suite	`TLS_AES_256_GCM_SHA384` (256-bit symmetric AEAD)
PKI Validation Engine	Python `ssl` with Mozilla Root CA bundle (`certifi`)
Verification Status	**VALID & TRUSTED** (`check_hostname=True, ssl.CERT_REQUIRED`)
Issuer Organization	`SSL.com EV Root Certification Authority RSA R2`
Subject Alternative Names	`cloudflare-dns.com, *.cloudflare-dns.com, one.one.one.one`
Certificate Serial Number	`4ED03304C46B87A8C2EB5569DB9EBA0C`
Validity Window	Dec 31 19:20:01 2025 GMT to Dec 21 19:20:01 2026 GMT

8.3 Evidence 3: Resilience Against Invalid Responses, Upstream Failures, and Timeouts

1. **Invalid / Corrupted Wire Payloads:** In automated test `test_invalid_upstream_response_handling`, the mock upstream returned random non-DNS bytes (`b'CORRUPTED_PAYLOAD'`). **cipherDNS** caught

`dns.message.DecodeError`, prevented crashes, and synthesized an RFC-compliant `SERVFAIL` (RCODE 2) response to the client.

2. **Upstream Blackhole Timeouts:** In automated test `test_upstream_timeout_handling`, queries targeted a non-routable address (`192.0.2.1:443`). The resolver caught `httpx.TimeoutException` after 3.5 s and returned a valid `SERVFAIL` packet, preserving the query ID.
3. **Firewall Connection Drops:** Where local firewalls drop packets to `8.8.8.8:443`, the client caught `httpx.ConnectError` and emitted `SERVFAIL` without interrupting the local UDP listener.

8.4 Evidence 4: Zeek Network Privacy Verification Matrix

Table 6 provides a side-by-side comparison of Zeek analysis across traditional DNS and **cipherDNS**.

Table 6: Comparative Zeek 9.0.0 Traffic Inspection Evidence Matrix

Inspection Metric	Traditional Plaintext DNS (UDP 53)	cipherDNS Encrypted DoH (TCP 443)
Domain Query Visibility	**EXPOSED:** `"netflix.com"` logged in cleartext in `dns.log`	**SHIELDED:** 0 entries in `dns.log`; completely empty
Resolved IP Visibility	**EXPOSED:** `207.45.72.1, 207.45.73.1` exposed in `dns.log`	**SHIELDED:** Zero IPs leaked across physical interface
Transport Protocol Logging	`proto=udp service=dns`	`proto=tcp service=ssl (TLS 1.3)`
TLS Handshake Evidence	*None (Unencrypted plaintext datagram)*	`ssl.log` confirms `TLS_AES_256_GCM_SHA384` session
Record Layer Protection	Cleartext RFC 1035 UDP datagram payload	**ContentType 23 (Application Data)** (AEAD encrypted)
Final Privacy Verdict	**FAIL: Critical Eavesdropping Breach**	**PASS: Cryptographically Secured (100%)**